

Занятие 3.3 Лабораторная работа.

Развертывание распределенного хранилища.

ЧАСТЬ 1. ЗАДАНИЕ НА ЛАБОРАТОРНУЮ РАБОТУ

Цель работы.

Освоить технологии развертывания распределенного хранилища данных на базе экосистемы Apache Hadoop и табличного формата Apache Iceberg с использованием фреймворка Apache Spark (PySpark). Получить навыки проектирования, настройки и первичного наполнения распределенного хранилища под конкретную предметную область.

Проверяемые компетенции.

Код	Индикатор	Уровень	
BD-4.1	Осуществляет выбор технологий обработки больших данных, приемлемых для создания прикладной системы ИИ с заданными требованиями	Средний	Способен организовывать распределенное хранилище и параллельную обработку на базе современных технологий (Hadoop, Spark) больших данных
PL-1.3	Разрабатывает и поддерживает системы обработки больших данных различной степени сложности	Средний	Способен применять основные функции фреймворка PySpark, может самостоятельно построить процесс обработки больших данных с использованием Airflow

Дескриптор освоения: Способен организовывать распределенное хранилище и параллельную обработку на базе современных технологий (Hadoop, Spark) больших данных. Осуществляет выбор технологий обработки больших данных, приемлемых для создания прикладной системы ИИ с заданными требованиями.

Задание

1. Подключиться к кластеру Hadoop/Spark через SSH, проверить доступность сервисов HDFS и YARN.
2. Спланировать архитектуру распределенного хранилища (выбрать стратегию организации директорий в HDFS, схему партиционирования, формат хранения).
3. Самостоятельно скачать «сырой» датасет с платформы Kaggle и загрузить его в HDFS с использованием консольного клиента `hdfs dfs`.
4. Настроить сессию Apache Spark с интеграцией Apache Iceberg (подключить необходимые пакеты, указать warehouse-путь).
5. Создать базу данных и «сырую» таблицу (Raw Layer) в формате Apache Iceberg.
6. Выполнить первичную очистку данных: приведение типов, обработка пропущенных значений, удаление дубликатов, фильтрация выбросов.
7. Провести разведочный анализ данных (EDA) с использованием PySpark DataFrame API.
8. Сформировать производные признаки (объединение бинарных признаков сертификации, расчёт возраста автомобиля).
9. Сохранить очищенные данные в целевую Iceberg-таблицу с применением выбранной стратегии партиционирования.
10. Проверить корректность развернутого хранилища: целостность данных, структуру партиций, доступ через Spark SQL.
11. Подготовить отчёт о проделанной работе.

Примечание: Полученная в результате выполнения работы Iceberg-таблица станет основой для выполнения следующей лабораторной работы (Занятие 3.4 — Формирование детального слоя данных DDS).

Варианты заданий (по номеру студента в группе mod 4)

Исходные данные:

Студенты самостоятельно скачивают исходный "сырой" датасет US Used Cars Dataset с платформы Kaggle по ссылке:

https://www.kaggle.com/datasets/ananyamital/us-used-cars-dataset?select=used_cars_data.csv

Общие этапы очистки для всех вариантов:

- Приведение типов (daysonmarket, maximum_seating, year → integer; horsepower, mileage, price → float/double; бинарные признаки → boolean)
- Преобразование major_options из строки в массив строк (через `regex_extract_all`)
- Удаление столбцов с большим количеством пропусков (>50%): fleet, has_accidents, frame_damaged, salvage
- Фильтрация vin по регулярному выражению `^[A-Z0-9]{17}$`
- Удаление дубликатов по vin
- Обработка пропусков: body_type → "Unknown", wheel_system → мода, horsepower → среднее, maximum_seating → среднее
- Ограничение выбросов: `daysonmarket <= 500, horsepower < 400, mileage <= 200000, price <= 60000, maximum_seating < 20`
- Формирование производного признака `is_any_cert = is_certified | is_cpo | is_oemcpo`
- Формирование производного признака `age = 2026 - year`

Различия между вариантами заключается в стратегии партиционирования итоговой Iceberg-таблицы и фокусе разведочного анализа.

Вариант 0. Хранилище с партиционированием по году и штату

Стратегия партиционирования Iceberg-таблицы: year, state

Обоснование: такая стратегия оптимальна для запросов, фильтрующих автомобили по году выпуска и региону продажи (типичный сценарий для аналитики рынка подержанных автомобилей).

Фокус EDA:

- Распределение количества автомобилей по штатам (топ-10)
- Распределение средней цены по годам выпуска
- Корреляция между year, mileage, price

Итоговая структура хранилища:

- Raw-таблица: used_cars_raw
- Clean-таблица: used_cars_clean

Партиционирование: years(year), state

Формат: Apache Iceberg

Вариант 1. Хранилище с партиционированием по типу кузова и возрасту

Стратегия партиционирования Iceberg-таблицы: body_type, age

Обоснование: такая стратегия оптимальна для запросов, анализирующих рынок по категориям автомобилей и их возрасту (типичный сценарий для сравнительного анализа сегментов рынка).

Фокус EDA:

- Распределение количества автомобилей по типам кузова
- Зависимость средней цены от возраста автомобиля для каждого body_type
- Доля сертифицированных автомобилей (is_any_cert) в разрезе body_type

Итоговая структура хранилища:

- Raw-таблица: used_cars_raw
- Clean-таблица: used_cars_clean

Партиционирование: body_type, age

Формат: Apache Iceberg

Вариант 2. Хранилище с партиционированием по типу привода и наличию сертификации

Стратегия партиционирования Iceberg-таблицы: wheel_system, is_any_cert

Обоснование: такая стратегия оптимальна для запросов, сравнивающих ценообразование и характеристики автомобилей с разным типом привода и сертификационным статусом (типичный сценарий для анализа премиальности и надёжности).

Фокус EDA:

- Распределение автомобилей по типам привода (AWD/FWD/RWD)
- Средняя цена и пробег в разрезе wheel_system и is_any_cert
- Доля сертифицированных автомобилей по штатам

Итоговая структура хранилища:

- Raw-таблица: used_cars_raw
- Clean-таблица: used_cars_clean

Партиционирование: wheel_system, is_any_cert

Формат: Apache Iceberg

Вариант 3. Хранилище с партиционированием по штату и бакетам пробега

Стратегия партиционирования Iceberg-таблицы: state, бакетизированный признак mileage_bucket

Обоснование: такая стратегия оптимальна для запросов, анализирующих рынок подержанных автомобилей по регионам и категориям пробега (типичный сценарий для оценки износа и региональных предпочтений).

Дополнительный этап: перед партиционированием студент создаёт признак mileage_bucket с помощью Bucketizer из pyspark.ml.feature (границы: 0, 50000, 100000, 150000, 200000).

Фокус EDA:

- Распределение автомобилей по бакетам пробега
- Средняя цена в разрезе state и mileage_bucket
- Зависимость мощности двигателя (horsepower) от пробега

Итоговая структура хранилища:

- Raw-таблица: used_cars_raw

- Clean-таблица: used_cars_clean

Партиционирование: state, mileage_bucket

Формат: Apache Iceberg

Контрольные вопросы

1. Что такое распределенное хранилище данных? В чем его отличие от локальной файловой системы?
2. Опишите архитектуру HDFS. Какую роль играют NameNode и DataNode? Что такое heartbeat и репликация?
3. Какие команды консольного клиента HDFS (hdfs dfs) используются для загрузки, выгрузки и просмотра файлов? Приведите примеры.
4. Что такое Apache YARN? Какие компоненты входят в его архитектуру (ResourceManager, NodeManager, ApplicationMaster)?
5. В чем отличие между RDD и DataFrame в Apache Spark? Что такое ленивые вычисления (lazy evaluation) и DAG?
6. Что такое Apache Iceberg? Какие преимущества он дает по сравнению с традиционными форматами хранения (Parquet, ORC)?
7. Что такое партиционирование и bucketing в контексте Iceberg-таблиц? Как они влияют на производительность запросов?
8. Как настроить Spark-сессию для работы с Iceberg? Какие параметры SparkConf необходимо указать?
9. Какие этапы включает процесс развертывания распределенного хранилища под конкретную предметную область?
10. Как проверить корректность развернутого хранилища и целостность загруженных данных?

ЧАСТЬ 2. МЕТОДИЧЕСКИЕ УКАЗАНИЯ

1. Теоретические основы

1.1. Распределенное хранилище данных и экосистема Hadoop

Распределенное хранилище данных представляет собой систему хранения, в которой данные размещаются на множестве узлов кластера и управляются как единое целое. Ключевые преимущества:

- Масштабируемость (горизонтальное добавление узлов)
- Отказоустойчивость (репликация данных)
- Высокая пропускная способность (параллельный доступ)

Apache Hadoop — фреймворк с открытым исходным кодом для распределенной обработки больших данных. Основные компоненты экосистемы:

- HDFS (Hadoop Distributed File System) — распределенная файловая система
- YARN (Yet Another Resource Negotiator) — менеджер ресурсов кластера
- MapReduce — модель пакетной обработки данных
- Hive, Spark, Iceberg — надстройки для работы со структурированными данными

1.2. Архитектура HDFS

HDFS построена по принципу master/slave:

- NameNode (master) хранит метаданные файловой системы (дерево каталогов, соответствие блоков файлам, расположение реплик)
- DataNode (slave) хранит фактические блоки данных, регулярно отправляет heartbeat на NameNode

Ключевые свойства HDFS:

- Репликация блоков данных (по умолчанию коэффициент 3)
- Поддержка только режима «записать один раз, читать много раз»
- Оптимизация под потоковую запись и чтение больших файлов

1.3. Архитектура YARN

YARN разделяет функции управления ресурсами и выполнения задач:

- ResourceManager — глобальный арбитр ресурсов кластера
- NodeManager — агент на каждом узле, управляющий контейнерами
- ApplicationMaster — компонент, отвечающий за конкретное приложение (например, Spark-приложение)

1.4. Apache Spark и интеграция с Iceberg

Apache Spark — фреймворк для распределенной обработки данных с поддержкой пакетной и потоковой обработки. Ключевые концепции:

- DataFrame — распределенная коллекция данных, организованная в именованные колонки
- Driver — координирует выполнение приложения
- Executor — выполняет задачи на рабочих узлах
- Lazy evaluation — отложенное выполнение операций до момента действия (action)

Apache Iceberg — открытый табличный формат для больших аналитических наборов данных. Ключевые возможности:

- ACID-транзакции
- Схемная эволюция
- Скрытое партиционирование
- Time Travel (доступ к историческим версиям)
- Интеграция с Spark, Trino, Flink

2. Подготовка окружения

2.1. Подключение к кластеру

Подключение к узлу кластера

```
ssh hadoop-node
```

Проверка статуса HDFS

```
hdfs dfsadmin -report
```



```
# Проверка статуса YARN
```

```
yarn node -list
```

```
# Переход в рабочую директорию
```

```
cd ~/student_workspace/lab3_3
```

```
mkdir -p warehouse
```

2.2. Загрузка данных в HDFS

```
# Создание директории для сырых данных
```

```
hdfs dfs -mkdir -p /user/$USER/datasets/raw
```

```
# Загрузка файла из локальной ФС в HDFS
```

```
# (файл used_cars_data.csv предварительно скачан с Kaggle)
```

```
hdfs dfs -put used_cars_data.csv /user/$USER/datasets/raw/
```

```
# Проверка загрузки
```

```
hdfs dfs -ls /user/$USER/datasets/raw/
```

```
# Просмотр первых строк файла
```

```
hdfs dfs -cat /user/$USER/datasets/raw/used_cars_data.csv | head -n 5
```

2.3. Настройка Spark-сессии с интеграцией Iceberg

```
from pyspark.sql import SparkSession
```

```
from pyspark import SparkConf
```

```
import os
```

```
def create_spark_configuration() -> SparkConf:
```

```
    """
```

```
    Создает и конфигурирует экземпляр SparkConf для приложения Spark  
    с интеграцией Apache Iceberg.
```

```
    """
```

```
    user_name = os.getenv("USER")
```

```
    conf = SparkConf()
```

```

conf.setAppName("Lab3_3_Distributed_Storage_Deployment")
conf.setMaster("yarn")
conf.set("spark.submit.deployMode", "client")
conf.set("spark.executor.memory", "8g")
conf.set("spark.executor.cores", "4")
conf.set("spark.executor.instances", "2")
conf.set("spark.driver.memory", "4g")

# Подключение Apache Iceberg
conf.set("spark.jars.packages",
        "org.apache.iceberg:iceberg-spark-runtime-3.5_2.12:1.6.0")
conf.set("spark.sql.extensions",
        "org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions")
conf.set("spark.sql.catalog.spark_catalog",
        "org.apache.iceberg.spark.SparkCatalog")
conf.set("spark.sql.catalog.spark_catalog.type", "hadoop")
conf.set("spark.sql.catalog.spark_catalog.warehouse",
        f"hdfs:///user/{user_name}/warehouse")
conf.set("spark.sql.catalog.spark_catalog.io-impl",
        "org.apache.iceberg.hadoop.HadoopFileIO")

return conf

```

```

conf = create_spark_configuration()
spark = SparkSession.builder.config(conf=conf).getOrCreate()

```

3. Ход работы

Этап 1. Планирование архитектуры хранилища

Перед началом работы студент должен спланировать структуру хранилища согласно своему варианту:

1. Определить структуру HDFS-директорий:

```

/user/<student>/warehouse/used_cars/
├── raw/      # сырые данные
└── clean/    # очищенные данные

```

2. Выбрать стратегию партиционирования (согласно варианту задания).
3. Определить схему данных — список колонок с типами.

Задание для студента: Составить схему хранилища в виде диаграммы или таблицы, обосновать выбор стратегии партиционирования.

Этап 2. Создание базы данных и сырой таблицы

```
# Создание базы данных
```

```
user_name = os.getenv("USER")
```

```
database_name = f"{user_name}_database"
```

```
spark.sql(f"CREATE DATABASE IF NOT EXISTS spark_catalog.{database_name}")
```

```
spark.catalog.setCurrentDatabase(database_name)
```

```
# Загрузка сырых данных из HDFS в DataFrame
```

```
raw_df = spark.read.format("csv") \
```

```
    .option("header", "true") \
```

```
    .option("inferSchema", "true") \
```

```
    .load(f"hdfs:///user/{user_name}/datasets/raw/used_cars_data.csv")
```

```
# Создание сырой Iceberg-таблицы
```

```
raw_df.writeTo("used_cars_raw").using("iceberg").createOrReplace()
```

```
# Проверка
```

```
print(f"Raw table count: {spark.table('used_cars_raw').count()}")
```

```
spark.table("used_cars_raw").printSchema()
```

Задание для студента: Адаптировать код под свою базу данных (использовать свою фамилию).

Этап 3. Первичная очистка данных

```
from pyspark.sql import functions as F
```

```
from pyspark.sql.types import DoubleType, IntegerType, BooleanType
```

```
from pyspark.sql.functions import regexp_extract_all, lit
```

```
# Приведение типов
```

```
cleaned_df = raw_df \
```

```
    .withColumn("daysonmarket", F.col("daysonmarket").cast(IntegerType())) \
```

```

.withColumn("horsepower", F.col("horsepower").cast(DoubleType())) \
.withColumn("mileage", F.col("mileage").cast(DoubleType())) \
.withColumn("price", F.col("price").cast(DoubleType())) \
.withColumn("maximum_seating", F.col("maximum_seating").cast(IntegerType())) \
.withColumn("year", F.col("year").cast(IntegerType()))

# Преобразование major_options из строки в массив
cleaned_df = cleaned_df.withColumn(
    "major_options",
    regexp_extract_all(F.col("major_options"), lit(r"([^\s]*)")), 1)
)

# Удаление столбцов с большим количеством пропусков
columns_to_drop = ["fleet", "has_accidents", "frame_damaged", "salvage"]
cleaned_df = cleaned_df.drop(*columns_to_drop)

# Фильтрация VIN по регулярному выражению
vin_pattern = r"^[A-Z0-9]{17}$"
cleaned_df = cleaned_df.filter(F.col("vin").rlike(vin_pattern))

# Удаление дубликатов по VIN
cleaned_df = cleaned_df.dropDuplicates(subset=["vin"])

# Обработка пропущенных значений
cleaned_df = cleaned_df.fillna({
    "body_type": "Unknown",
    "wheel_system": "AWD" # мода
})

# Ограничение выбросов
cleaned_df = cleaned_df \
    .filter(F.col("horsepower") < 400) \
    .filter(F.col("mileage") <= 200000) \
    .filter(F.col("price") <= 60000) \
    .filter(F.col("maximum_seating") < 20) \

```

```

.withColumn("daysonmarket",
            F.when(F.col("daysonmarket") > 500, 500)
            .otherwise(F.col("daysonmarket")))

# Формирование производных признаков
cleaned_df = cleaned_df.withColumn(
    "is_any_cert",
    F.col("is_certified") | F.col("is_cpo") | F.col("is_oemcpo")
).withColumn(
    "age",
    2024 - F.col("year")
)

```

Задание для студента: Реализовать стратегию очистки согласно своему варианту, обосновать выбор методов обработки пропущенных значений и выбросов.

Этап 4. Разведочный анализ данных (EDA)

```

# Базовая статистика
cleaned_df.describe().show()

# Анализ распределения категориальных признаков
cleaned_df.groupBy("body_type").count().orderBy(F.col("count").desc()).show(10)

# Анализ пропущенных значений
null_counts = cleaned_df.select([
    F.sum(F.when(F.col(c).isNull(), 1).otherwise(0)).alias(c)
    for c in cleaned_df.columns
])
null_counts.show()

# Анализ выбросов (для числовых признаков)
cleaned_df.select("price", "mileage", "horsepower").summary().show()

```

Задание для студента: Провести EDA согласно фокусу своего варианта (см. раздел «Варианты заданий»), выявить аномалии и особенности данных.

Этап 5. Загрузка в целевое хранилище с партиционированием

Пример для Варианта 0 (партиционирование по year и state):

```
(cleaned_df
    .writeTo("used_cars_clean")
    .using("iceberg")
    .tableProperty("format-version", "2")
    .tableProperty("write.parquet.compression-codec", "zstd")
    .partitionedBy(F.col("year"), F.col("state"))
    .createOrReplace()
)
```

Пример для Варианта 3 (с предварительной бакетизацией mileage):

```
from pyspark.ml.feature import Bucketizer

bucketizer = Bucketizer(
    splits=[0, 50000, 100000, 150000, 200000],
    inputCol="mileage",
    outputCol="mileage_bucket"
)

cleaned_df_bucketed = bucketizer.transform(cleaned_df)

(cleaned_df_bucketed
    .writeTo("used_cars_clean")
    .using("iceberg")
    .tableProperty("format-version", "2")
    .partitionedBy(F.col("state"), F.col("mileage_bucket"))
    .createOrReplace()
)
```

Задание для студента: Реализовать запись согласно стратегии партиционирования своего варианта.

Этап 6. Проверка корректности хранилища

```
# Проверка целостности данных
raw_count = spark.table("used_cars_raw").count()
clean_count = spark.table("used_cars_clean").count()
print(f"Raw count: {raw_count}, Clean count: {clean_count}")
```

```
# Проверка схемы
spark.table("used_cars_clean").printSchema()

# Проверка доступности через Spark SQL
spark.sql("SELECT * FROM used_cars_clean LIMIT 5").show()

# Проверка структуры партиций
spark.sql("""
    SELECT year, state, count(*) as records_count
    FROM used_cars_clean
    GROUP BY year, state
    ORDER BY records_count DESC
    LIMIT 20
    """).show()

# Проверка метаданных Iceberg
spark.sql("SELECT * FROM used_cars_clean.history").show()
```

4. Требования к отчёту

Отчёт должен содержать:

1. Титульный лист (ФИО студента, группа, дата выполнения, вариант задания)
2. Цель и задачи работы
3. Спроектированная архитектура хранилища:
 - Структура HDFS-директорий
 - Схема данных (список колонок с типами)
 - Обоснование стратегии партиционирования
4. Описание исходных данных (объем, характеристики)
5. Реализованные этапы очистки данных (с обоснованием выбора методов, включая пороги фильтрации выбросов)

6. Результаты разведочного анализа (статистика, распределения, аномалии — согласно фокусу варианта)

7. Результаты развертывания хранилища:

- Скриншоты структуры партиций
- Подтверждение целостности данных
- Результаты проверки через Spark SQL

8. Код выполнения работы (Jupyter Notebook)

9. Выводы по результатам работы

10. Ответы на контрольные вопросы

Формат сдачи:

- Jupyter Notebook (.ipynb) с выполненным кодом
- Отчёт в формате PDF или Markdown
- Загрузка в Git-репозиторий

1. Критерии оценивания

Критерий	Баллы	Описание
Проектирование архитектуры хранилища	15	Обоснованная структура HDFS и стратегия партиционирования
Настройка окружения	10	Корректная настройка Spark-сессии с Iceberg
Загрузка данных в HDFS	10	Использование консольного клиента hdfs dfs
Создание базы данных и raw-таблицы	10	Корректное создание Iceberg-таблицы
Первичная очистка данных	15	Обработка пропусков, приведение типов, удаление дубликатов, фильтрация выбросов
Разведочный анализ (EDA)	15	Статистика, распределения, выявление аномалий (согласно фокусу варианта)
Загрузка в clean-таблицу с партиционированием	10	Корректная запись с применением партиционирования
Проверка корректности хранилища	5	Подтверждение целостности и доступности данных
Отчет	5	Полнота, структура, выводы
Защита (ответы на вопросы)	5	Понимание теоретических основ
Итого	100	

Шкала оценивания:

- 90-100 баллов: «Отлично»
- 76-89 баллов: «Хорошо»
- 61-75 баллов: «Удовлетворительно»

- Менее 61 балла: «Неудовлетворительно»

6. Правила использования внешних ресурсов и генеративного ИИ

Разрешено:

- Использование официальной документации Apache Hadoop, Spark, Iceberg
 - Консультации с генеративным ИИ (ChatGPT, YandexGPT, GigaChat)
- для:

- Объяснения синтаксиса PySpark и HDFS-команд
- Поиска оптимальных стратегий партиционирования
- Генерации примеров кода для ETL-процессов
- Проверки логики реализации

Запрещено:

- Полная генерация кода ИИ без понимания и модификации
- Копирование решений других студентов
- Использование готовых решений из открытых репозиториях без адаптации

Требования:

- Все запросы к ИИ должны быть задокументированы в отчете
- Студент должен объяснить логику каждого этапа работы
- При защите необходимо продемонстрировать понимание реализованного кода и архитектуры хранилища

7. Список литературы

1. Захария М., Венделл П., Конвински Э., Карау Х. Изучаем Spark. Молниеносный анализ данных. — М.: ДМК Пресс, 2015. — 400 с.
2. Ральф Кимбалл, Марджи Росс. Инструментарий хранения и анализа данных. Полное руководство по размерному моделированию. — М.: Бомбора, 2021. — 656 с.
3. B. Harenslak, J. de Ruiter. Data Pipelines with Apache Airflow. — New York: Manning Publications, 2021. — 480 p.

4. Клеппманн М. Высоконагруженные приложения. Программирование, масштабирование, поддержка. — Астана: Sprint Book, 2024. — 640 с.

5. Официальная документация Apache Spark [Электронный ресурс]. — URL: <https://spark.apache.org/docs/latest/>

6. Официальная документация Apache Iceberg [Электронный ресурс]. — URL: <https://iceberg.apache.org/docs/latest/>

8. Необходимые гиперссылки для лабораторной работы.

1. Датасет:

- US Used Cars Dataset: https://www.kaggle.com/datasets/ananyamital/us-used-cars-dataset?select=used_cars_data.csv

2. Документация по технологиям:

- Apache Spark (официальная документация): <https://spark.apache.org/docs/latest/>

- PySpark API (Python): <https://spark.apache.org/docs/latest/api/python/>

- Apache Iceberg (официальная документация): <https://iceberg.apache.org/docs/latest/>

- Apache Hadoop (официальная документация): <https://hadoop.apache.org/docs/>

3. Для локального разворачивания окружения

Docker-образы:

- Apache Spark: <https://hub.docker.com/search?q=spark&type=image>

- Hadoop: <https://hub.docker.com/search?q=hadoop&type=image>

- Apache Iceberg: <https://hub.docker.com/search?q=iceberg&type=image>

Готовые решения для локального запуска:

- Spark + Iceberg в Docker: <https://github.com/apache/iceberg/tree/master/examples>

- Локальный кластер Hadoop: <https://github.com/big-data-europe/docker-hadoop>

4. Инструменты мониторинга и работы с HDFS (ключевые для данного занятия):

- Справочник команд HDFS Shell (HDFS Commands Guide): <https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-common/FileSystemShell.html>

- Документация по веб-интерфейсу YARN ResourceManager (для проверки статуса кластера): <https://hadoop.apache.org/docs/current/hadoop-yarn/hadoop-yarn-site/ResourceManagerRest.html>

- Документация по Spark Web UI (для мониторинга выполнения задач): <https://spark.apache.org>

Примечания для преподавателя

Методические рекомендации по организации занятия

1. Подготовительный этап (до занятия):

- Проверить доступность кластера Hadoop/Spark (HDFS, YARN)
- Подготовить рабочие директории для каждого студента в HDFS
- Убедиться в наличии пакетов Iceberg в Spark-окружении
- Предупредить студентов о необходимости заранее скачать датасет с Kaggle

2. Вводная часть (0.5 часа):

- Краткий обзор архитектуры HDFS и YARN
- Демонстрация примера развертывания хранилища
- Разбор типичных ошибок при работе с HDFS
- Важно: подчеркнуть связь с следующей лабораторной работой

(Занятие 3.4)

3. Основная часть (3 часа):

- Самостоятельная работа студентов

- Консультации по индивидуальным вопросам
- Промежуточная проверка архитектуры хранилища

4. Заключительная часть (0.5 часа):

- Демонстрация результатов
- Обсуждение проблемных моментов
- Анонс следующего занятия

Типичные ошибки студентов

1. Неправильная настройка SparkConf — отсутствие пакетов Iceberg, некорректные пути в HDFS
2. Неверная структура HDFS-директорий — создание файлов в корне вместо пользовательской директории
3. Игнорирование приведения типов — работа со строками вместо числовых/временных типов
4. Необоснованный выбор партиционирования — слишком мелкое или крупное партиционирование
5. Отсутствие проверки целостности — не проверяется количество записей до и после очистки
6. Неправильная обработка `major_options` — попытка работать со строкой как с массивом без `regex_extract_all`

Дополнительные задания для продвинутых студентов

1. Реализовать автоматическое создание структуры хранилища через скрипт
2. Добавить мониторинг размера партиций и оптимизировать стратегию партиционирования
3. Реализовать версионирование данных с использованием Iceberg Time Travel
4. Настроить сжатие данных (`zstd`, `snappy`) и оценить влияние на производительность
5. Подготовить презентацию архитектуры хранилища для защиты